

Part 1: Lists

1. The Shopping Cart

Goal: Perform basic operations: creation, adding, removing, and accessing items by index.

The Problem:

1. Create a list of strings called `cart` with the items: "Milk", "Eggs", "Bread".
 2. Add "Butter" to the end of the list.
 3. Remove "Eggs" from the list.
 4. Print the **first** item in the list.
-

2. The Price Filter

Goal: Filter a list based on a condition to create a new list.

The Problem:

Given a list of integer prices: [12, 45, 6, 89, 10, 23].

Write code that creates a new list called `expensiveItems`. Iterate through the original prices and add only prices greater than 20 to the new list. Print the `expensiveItems` list.

Part 2: Sets

1. The Duplicate Remover

Goal: Understand that Sets automatically enforce uniqueness.

The Problem:

You have a list of email addresses where some were entered twice:

emails = ['a@b.com', 'x@y.com', 'a@b.com', 'z@z.com'].

Convert this list into a Set to remove the duplicates automatically, then convert it back to a List (optional) or just print the Set and its size (length).

2. The Class Reunion

Goal: Use Set operations to compare two groups of data.

The Problem:

You have two sets of students:

1. mathClass = {'Alice', 'Bob', 'Charlie', 'David'}
2. scienceClass = {'Charlie', 'David', 'Eve', 'Frank'}

Find and print:

- **Both:** Students attending *both* classes (Intersection).
- **Math Only:** Students in Math who are *not* in Science (Difference).

Part 3: Maps

1. The User Profile

Goal: Create a map, access values using keys, and update a value.

The Problem:

1. Create a map called user with these key-value pairs:
 - 'username': 'admin'
 - 'age': 25
 - 'isAdmin': true
 2. Update the 'age' to 26.
 3. Print the sentence: "User [username] is [age] years old." (Replace brackets with actual values from the map).
-

6. The Frequency Counter

Goal: Use a Map to count how many times items appear in a List.

The Problem:

Given a list of votes: ['Red', 'Blue', 'Red', 'Green', 'Red', 'Blue'].

Create an empty Map<String, int> called voteCounts.

- **Key:** vote name
- **Value:** vote count

Part 4: Functions

1. Custom Sorting & Filtering (The Leaderboard)

Goal: Create a function that sorts a list of maps based on multiple criteria (Primary and Secondary sorting).

The Problem:

You have a List of Maps representing players, each with a 'name' (String) and 'score' (int) (e.g. {'name': 'Ahmed', 'score': 85}).

1. Filter out players with a score less than 50.
 2. Sort the remaining players by **Score (Descending)**.
 3. **Tie-breaker:** If two players have the same score, sort them by **Name (Alphabetical Ascending)**.
-

2. Data Grouping (The Categorizer)

Goal: Create a function to convert a flat List of Maps into a Map of Lists (Grouping data by a key).

The Problem:

You have a raw list of transactions. Each transaction has a category and an amount.

Write code to transform this list into a Map<String, List<double>> where the Key is the category and the Value is a list of all amounts belonging to that category.

e.g.

Input:

```
List<Map<String, dynamic>> transactions = [  
    {'category': 'Food', 'amount': 10.0},  
    {'category': 'Transport', 'amount': 5.0},  
    {'category': 'Food', 'amount': 20.0},  
    {'category': 'Shopping', 'amount': 50.0},  
    {'category': 'Transport', 'amount': 15.0},  
];
```

Result:

```
{  
    'Food': [10.0, 20.0],  
    'Transport': [5.0, 15.0],  
    'Shopping': [50.0],  
}
```